DOC-SMCTA-GOV-006 · VERSIÓN 2.1.0

Lecciones Aprendidas y Bitácora Forense de Ingeniería

Análisis Post-Mortem de Anomalías Críticas Resueltas bajo Estándar IEEE 1044 y Reglas de No-Regresión

Proyecto / Sistema	**Memoria Técnica Institucional y Aseguramiento Determinista de Calidad**
Fase / Área Normativa	**06 — Gobernanza de Ingeniería y Blindaje Técnico Oficial**
Autoría y Gobernanza	**Equipo de Ingeniería Backend, Frontend y QA — Proyecto SMCTA**
Fecha de Emisión	**Noviembre 2026**

Resumen

La diferencia entre un prototipo académico frágil y un sistema de grado transaccional radica en cómo responde ante condiciones borde: carreras de hilos en un mismo milisegundo, redondeos de comisiones compuestas, desconexiones a mitad de transacción y reintentos maliciosos. Esta bitácora documenta con transparencia forense las **seis anomalías técnicas reales (`INC-01` a `INC-06`)** detectadas, diagnosticadas y erradicadas durante la construcción y endurecimiento de **SMCTA v2**, dejando constancia exacta de su causa raíz, su prueba de regresión automatizada y la regla obligatoria para evitar que cualquier integrante vuelva a introducirlas.

Índice General de Contenidos

Resumen

Catálogo Consolidado de Incidentes Resueltos en Desarrollo

Análisis Forense Detallado por Anomalía

- INC-01: Doble Descuento de `platformShare` en la Conciliación de Ingresos del Comercio
- INC-02: Entrelazamiento Transaccional sobre Conexión Única en `PGlite` WASM
- INC-03: Despacho Prematuro de Eventos SSE Antes de Confirmar `COMMIT`
- INC-04: Ventana Residual de Fraude por Captura Reciente (*Ataque Boomerang*)
- INC-05: Liberación No Propietaria de Candados Distribuidos ante Pausas de Proceso
- INC-06: Inicialización Destructiva de Esquema y Mutabilidad Implícita del Libro Mayor

Síntesis de Disciplina Operativa para Nuevas Contribuciones del Equipo

Catálogo Consolidado de Incidentes Resueltos en Desarrollo

ID	Clasificación IEEE 1044	Subsistema Afectado	Impacto Potencial Pre-Corrección	Commit de Solución	Test Guardián
INC-01	Defecto Lógico / Contable	`escrowService.ts`, `p2pService.ts`	Subestimación sistemática de `tenantNetFees` por doble resta de `platformShare`	`9d9d8bf`	`securityAndRules.test.ts`

ID	Clasificación IEEE 1044	Subsistema Afectado	Impacto Potencial Pre-Corrección	Commit de Solución	Test Guardián
INC-02	Condición de Carrera I/O	`db/pglite.ts`	Error `cannot start a transaction within a transaction` bajo peticiones paralelas	`4641e78`	`qa02-concurrency.spec.ts`
INC-03	Consistencia Eventual / UI	`infra/eventHub.ts`, `services/*`	Notificaciones SSE "fantasma" emitidas antes de un potencial `ROLLBACK` SQL	`0b1f251`	`sseAndCors.test.ts`
INC-04	Vulnerabilidad de Replay	`qrToken.ts`, `couponRepo.ts`	Ataque *boomerang*: uso de captura QR tomada < 30 s antes de vender en P2P	`9d9d8bf`	`qrToken.test.ts`
INC-05	Exclusión Mutua Distribuida	`infra/lockManager.ts`	Borrado accidental de candado Redis ajeno tras pausa GC > TTL	`fad5b10`	`lockManager.test.ts`
INC-06	Destrucción de Datos / DDL	`db/schema.sql`	Pérdida total de tablas al reiniciar proceso y ausencia de freno a `UPDATE` en ledger	`4641e78`	`dbIntegrity.test.ts`

Análisis Forense Detallado por Anomalía

INC-01: Doble Descuento de `platformShare` en la Conciliación de Ingresos del Comercio

- **Síntoma y Causa Raíz:** Al registrar un asiento `P2P_SPLIT` en `escrow_ledgers`, `p2pService` almacenaba correctamente en `tenant_fee_accumulated` la comisión **ya neta** del comercio (`grossFee - platformShare`) y en `platform_fee` la porción de la plataforma. Sin embargo, en una versión preliminar del agregador de auditoría (`escrowService`), la consulta volvía a restar `platform_fee` sobre `SUM(tenant_fee_accumulated)`, descontando dos veces la participación de la plataforma SaaS y subreportando la ganancia real del cliente B2B.
- **Solución Implementada (`9d9d8bf`):** Definición canónica única en `Glosario Maestro`: `tenant_fee_accumulated` representa invariablemente el neto devengado por el *Tenant*, mientras que `grossFees = tenantNetFees + platformFees`. Se incorporó una aserción algebraica explícita en `securityAndRules.test.ts` verificando al centavo que `tenantNetFees + platformFees === sellerFeeAmount + buyerFeeAmount`.

- **Regla de No-Regresión:** Ningún servicio externo a `@smcta/shared/src/pricing.ts` puede recalcular porcentajes de reparto manualmente; debe consumir exclusivamente el objeto retornado por `calculateCommissionBreakdown`.

INC-02: Entrelazamiento Transaccional sobre Conexión Única en `PGlite` WASM

- **Síntoma y Causa Raíz:** A diferencia de `pg.Pool` (que entrega un socket TCP independiente a cada `BEGIN`), `@electric-sql/pglite` ejecuta una instancia embebida monohilo de PostgreSQL en memoria/disco sobre una única sesión virtual. Cuando `Promise.all` disparaba 50 compras simultáneas en `QA-02`, dos corrutinas asíncronas de Node.js intercalaban `BEGIN ISOLATION LEVEL SERIALIZABLE` sobre la misma sesión activa, provocando fallos de protocolo transaccional.
- **Solución Implementada (`4641e78`):** En `apps/backend/src/db/pglite.ts` se encapsuló el acceso en una cola determinista de promesas (`mutexChain`) que serializa de forma estricta cada bloque `withTransaction` y cada consulta individual sin alterar en absoluto la interfaz pública del puerto `Database`. En `PgDatabase` (`pg.ts`), por el contrario, el paralelismo real recae sobre el pool nativo de conexiones de PostgreSQL 15.
- **Regla de No-Regresión:** Nunca invocar `this.db.query` crudo eludiendo el encolador interno dentro del adaptador embebido `PGliteDatabase`.

INC-03: Despacho Prematuro de Eventos SSE Antes de Confirmar `COMMIT`

- **Síntoma y Causa Raíz:** Un error clásico al integrar canales en tiempo real (*WebSockets* o *Server-Sent Events*) consiste en llamar a `eventHub.publish(...)` en la última línea **dentro** del callback de `db.withTransaction(async (tx) ⇒ { ... })`. En `SERIALIZABLE`, PostgreSQL puede detectar una dependencia cíclica `rw-antidependency` recién en el instante exacto de ejecutar la sentencia `COMMIT`. Si el evento SSE ya había salido por red hacia el navegador, el usuario veía un cartel verde de *"¡Compra exitosa!"* sobre una operación que la base de datos acababa de revertir.
- **Solución Implementada (`0b1f251`):** Todos los servicios de aplicación (`checkoutService`, `p2pService`, `tpvService`) retornan el resultado junto con los eventos pendientes fuera del ámbito de `withTransaction`, ejecutando `eventHub.publish` estrictamente en las líneas posteriores a la resolución exitosa del `COMMIT`.

INC-04: Ventana Residual de Fraude por Captura Reciente (*Ataque Boomerang*)

- **Síntoma y Causa Raíz:** Con un TTL de 30 segundos en el QR dinámico, un atacante parado a cinco metros del control de acceso podía abrir su QR (`t = 0 s`), tomar una

captura desde otro teléfono, confirmar la venta o publicación del cupón en el mercado secundario (`t = 4 s`) y presentar la captura en el lector TPV (`t = 9 s`). Como la firma `HMAC-SHA256` era auténtica y no habían transcurrido 30 segundos, una validación puramente criptográfico-temporal sin estado habría sido engañada si solo miraba el payload o no ligaba la emisión al ciclo de tenencia.

- **Solución Implementada (`9d9d8bf`):** Adición de la columna `token_version INT NOT NULL DEFAULT 0` en `coupons` , incluida dentro del mensaje autenticado por `HMAC-SHA256` e incrementada en cada mutación de estado o titularidad. El TPV exige `coupon.tokenVersion === payload.version` bajo bloqueo `FOR UPDATE` , anulando instantáneamente el ataque con `409 QR_STALE_TOKEN` .

INC-05: Liberación No Propietaria de Candados Distribuidos ante Pausas de Proceso

- **Síntoma y Causa Raíz:** Si el Proceso A adquiere un candado con `TTL = 3000 ms` , sufre una pausa inesperada de red de `3500 ms` (expirando la clave en Redis), el Proceso B adquiere legítimamente el mismo candado en `t = 3100 ms` , y en `t = 3500 ms` el bloque `finally` del Proceso A ejecuta un `DEL key` ciego, el Proceso A destruye el candado activo del Proceso B abriendo la puerta a una carrera crítica triple (patrón documentado por Martin Kleppmann).
- **Solución Implementada (`fad5b10`):** Generación de `owner = randomUUID()` único por adquisición tanto en `RedisLockManager` (liberado exclusivamente mediante script Lua condicional en el servidor Redis) como en `InMemoryLockManager` (con mapa acotado y desalojo automático de entradas vencidas para evitar fugas de memoria).

INC-06: Inicialización Destructiva de Esquema y Mutabilidad Implícita del Libro Mayor

- **Síntoma y Causa Raíz:** Los scripts académicos tempranos solían comenzar con `DROP TABLE IF EXISTS ... CASCADE` para facilitar pruebas manuales y carecían de frenos en base de datos contra `UPDATE escrow_ledgers SET ...` . Ejecutar accidentalmente ese arranque contra una base compartida destruiría todo el historial financiero.
- **Solución Implementada (`4641e78`):** Reescritura íntegra de `schema.sql` empleando bloques idempotentes `DO $$ BEGIN ... EXCEPTION WHEN duplicate_object THEN NULL; END $$` , `CREATE TABLE IF NOT EXISTS` , `ON DELETE RESTRICT` en todas las claves foráneas y creación del disparador PL/pgSQL `trg_escrow_ledgers_immutable` que rechaza cualquier intento de mutación o borrado con `SQLSTATE 28000` (validado explícitamente en `dbIntegrity.test.ts`).

Síntesis de Disciplina Operativa para Nuevas Contribuciones del Equipo

1. **Prohibido usar `number` nativo para aritmética monetaria:** Todo cálculo de precios, bandas o comisiones debe delegarse en `packages/shared/src/money.ts` y `pricing.ts`.
2. **Prohibido superar $CC = 5$ por función:** Antes de abrir un *Pull Request*, todo integrante debe ejecutar `npm run lint:complexity` y descomponer funciones extensas en predicados o mapeadores puros con responsabilidad única.
3. **Prohibido emitir efectos secundarios externos dentro de `withTransaction`:** Ni llamadas HTTP a terceros ni envíos por `EventHub` pueden ocurrir antes de completarse el `COMMIT`.
4. **Prohibido serializar secretos:** Toda nueva credencial criptográfica de *Tenant* debe envolverse en un objeto de valor opaco análogo a `QrSecret`.